

Vorlesung 3: Architektur

Prof. Dr. Falk Howar

Automated Quality Assurance
Lehrstuhl für Software Engineering
Fakultät für Informatik
TU Dortmund



Architektur.

- Die 4+1 Sichten auf Architektur **aufzählen**, **erläutern** und damit eine Architektur **beschreiben**.



- Treiber für Architektur-Entscheidungen **verstehen**



- Qualitätskriterien auf Architekturen **anwenden**, um diese zu **vergleichen**



Integration und Daten-Intensive Anwendungen.

- Pattern Integration u. Skalierung **kennen** und **bewerten** können



- Batch Datenverarbeitung **verstehen** und **erläutern**



ADD 3.0.

- Begriffe des Architektur Design **benennen** und **erläutern** können



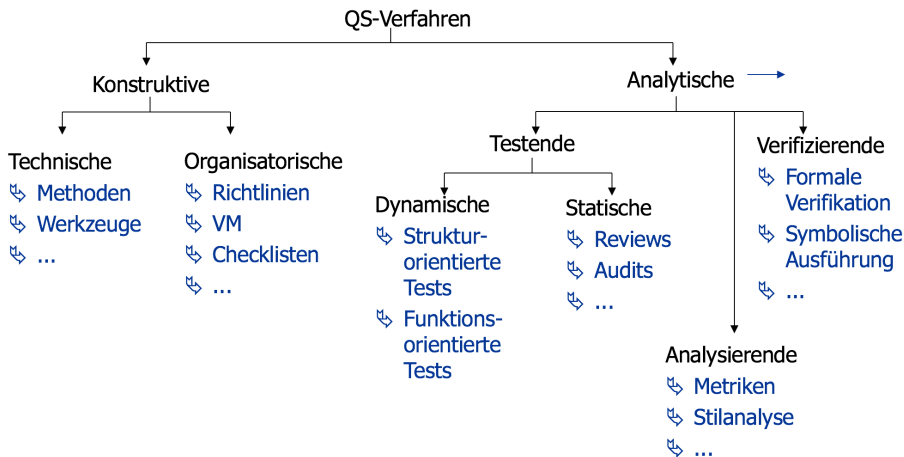
- ADD 3.0 Vorgehen **anwenden** können



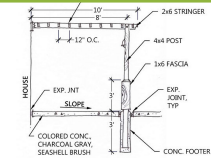
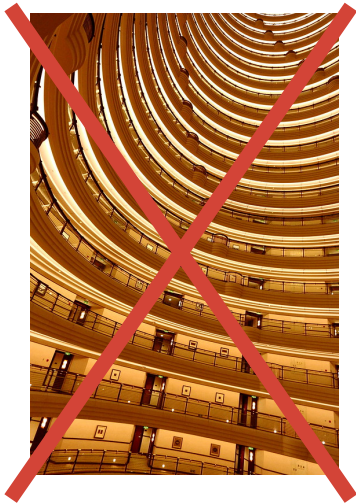
Agenda

1. Motivation
2. Begriffe und Definitionen
3. Dokumentation: 4+1 Sichten
4. Verteilte und Datenintensive Anwendungen
5. Agiles Architektur Design: ADD 3.0
6. Zusammenfassung
7. Literatur und Standards

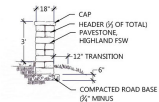
Klassifizierung von Ansätzen



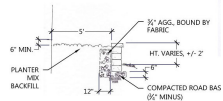
Analogien "Architektur"



① WOOD ARBOR



② FREE STANDING WALL



③ RETAINING WALL

<https://flic.kr/p/9FbvPx> (CC BY)

<https://flic.kr/p/9mafXs> (CC BY)

Analogien “Architektur”

Dokumentation:

- *“Andere Disziplinen, wie Maschinenbau, Immobilienarchitektur und Elektrotechnik, haben ungeheuer von der Standardisierung ihrer jeweiligen Pläne profitiert.”*
- *“Heute kann jeder Ingenieur die Konstruktionszeichnungen seiner Berufskollegen verstehen und weiterentwickeln.”*

Konstruktionsprinzipien unterstützen Ziele:

- Wir wissen aus Erfahrung und Versuchen, wie eine Brücke konstruiert sein muss, um bestimmte Belastungen auszuhalten
- Wir haben Muster, nach denen wir Verkehrskreuzungen und Autobahnkreuze kosten-effizient, sicher, effizient, usw. gestalten.

Zitate: G. Starke, P. Hruschka: Software-Architektur kompakt — angemessen und zielorientiert (2. Auflage), 2011

Beispiel: Remote Home Control



Ideen für Architektur?

Abschnitt	Inhalt
Bezeichner	UC-1
Name	Gerät schalten
Autoren	F. Howar
Priorität	...
Quelle	...
Verantwortlich	...
Kurz-Beschreibung	Benutzer schaltet ein Gerät von unterwegs ein.
Detaillierte Beschreibung	Benutzer wünscht, ein Gerät zu schalten.
Voraussetzung	App installiert und Gerät registriert.
Nachbedingung	
Ergebnis	
Hauptszenario	App öffnen, Gerät aus Übersicht wählen, Gerät schalten
Alternativ-Szenarien	...
Ausnahme-Szenarien	...
Einzelne Anforderungen	REQ-F-10, ...
Qualitäten	REQ-Q-1, REQ-Q-2, ...

Beispiel: Remote Home Control

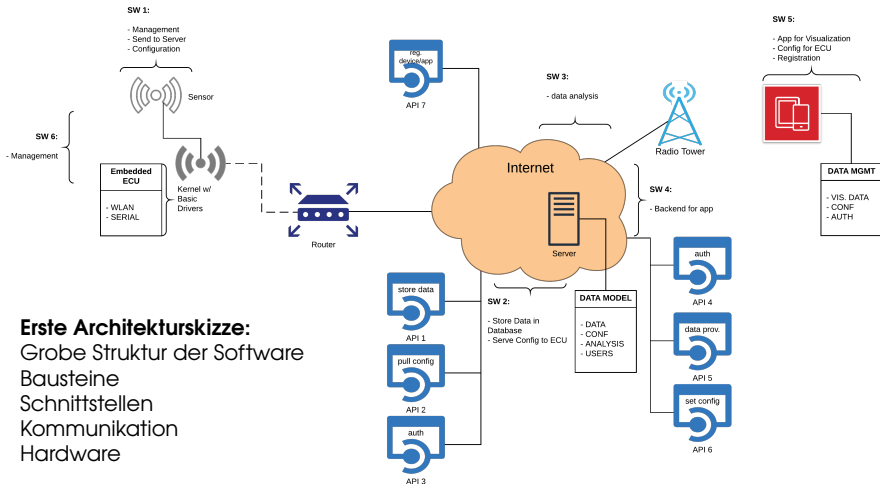
Fragen:

- Wie werden Systeme im Haus angebunden?
- Wie kommuniziert die App mit den Systemen?
- Wie erhält ein Nutzer Benachrichtigungen?

Herausforderungen und Ziele:

- Verteiltes System
- Sicherheit

Beispiel: Remote Home Control



Erste Architekturskizze:

Grobe Struktur der Software

Bausteine

Schnittstellen

Kommunikation

Hardware

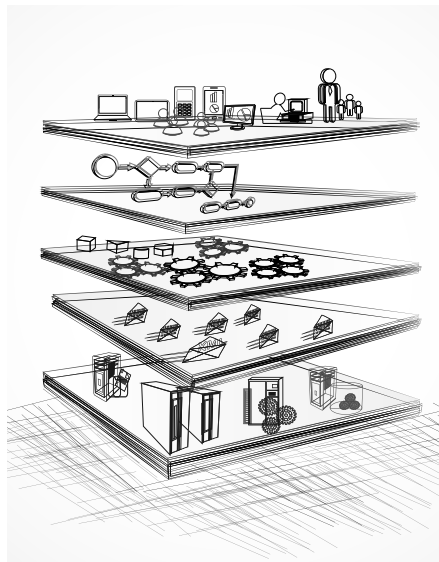
Software Architektur

Ziele:

- Grobe Struktur, die bestimmte (nicht-funktionale) Anforderungen erfüllt
- Dokumentation der groben Struktur

Aspekte:

- Logische Struktur (z.B. Teil-Systeme, Dienste, Komponenten, Schnittstellen, ...)
- Dynamische Aspekte: Abläufe im System, Kommunikation zwischen Teilen
- Verteilung von Software auf Hardware
- Struktur für die Entwicklung (Code, Artefakte)



Quelle: Shutterstock

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Dokumentation: 4+1 Sichten
4. Verteilte und Datenintensive Anwendungen
5. Agiles Architektur Design: ADD 3.0
6. Zusammenfassung
7. Literatur und Standards

Software Architektur

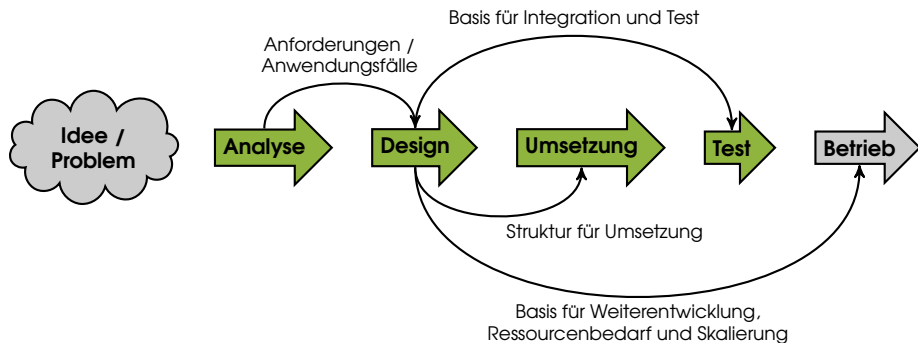
Definition (Software Architektur)

- ① Die Architektur eines Systems beschreibt die Strukturen des Systems, dessen *Bausteine*, *Schnittstellen* und deren Zusammenspiel.
- ② Die Architektur besteht aus Plänen und enthält meistens Vorschriften oder Hinweise, wie das System erstellt oder zusammengebaut werden sollte.

Als Plan unterstützt die Architektur ...

- die Herstellung (hier: Programmierung und Test)
- die Weiterentwicklung und den Einsatz des Systems (hier: Betrieb)

Architektur im Entwicklungsprozess



Bausteine und Schnittstellen

Definition (Bausteine)

Bausteine sind unabhängige, ersetzbare **Teile** mit wohldefinierten Aufgaben und **Schnittstellen** (Interfaces), welche inneren Zustand und Verhalten **kapseln**.

Beispiele für Bausteine:

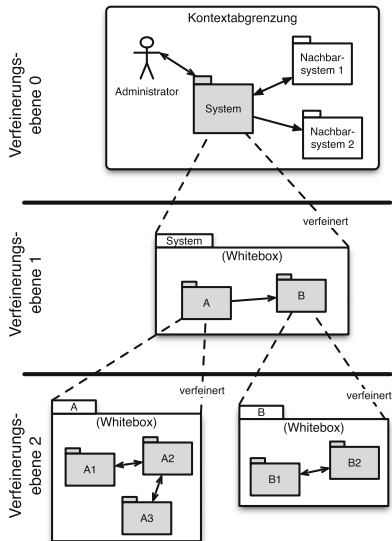
- Klassen
- Komponenten
- Dienste
- Teil-Systeme

Definition (Schnittstellen)

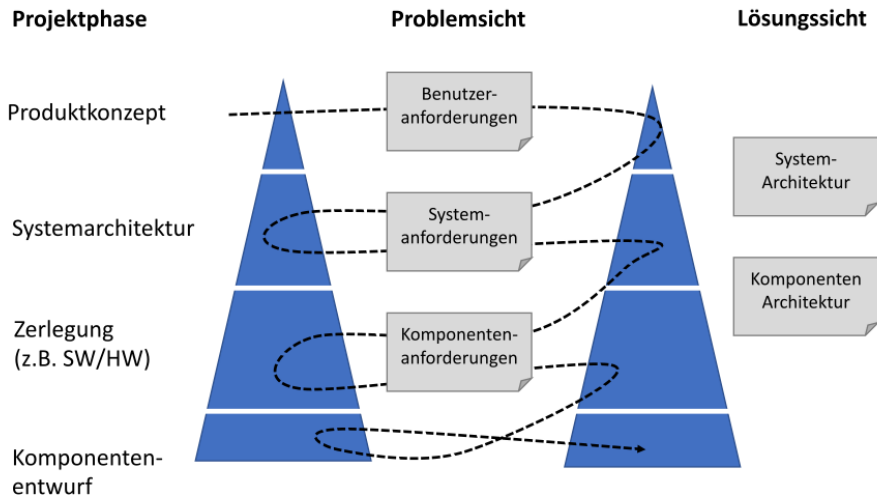
Schnittstellen sind definierte Interaktionspunkte (= Methoden, Datenformat und Protokoll) zwischen Teilen eines Softwaresystems.

Hierarchische Komposition und Präzisierung

- Software-Architekturen bestehen aus Bausteinen
- Bausteine tragen Verantwortung und kommunizieren über Schnittstellen
- **Blackboxes** sind nur durch ihre Schnittstellen und Funktionen charakterisiert. Sie verstecken ihr gesamtes Innenleben.
- **Whiteboxes** präzisieren Blackboxes durch das Angeben innerer Struktur.



Ko-Entwicklung von Architektur und Anforderungen



nach Ebert, 2014: Systematisches Requirements Engineering, 5. Auflage

Wie kommen wir zu guten Architekturen?

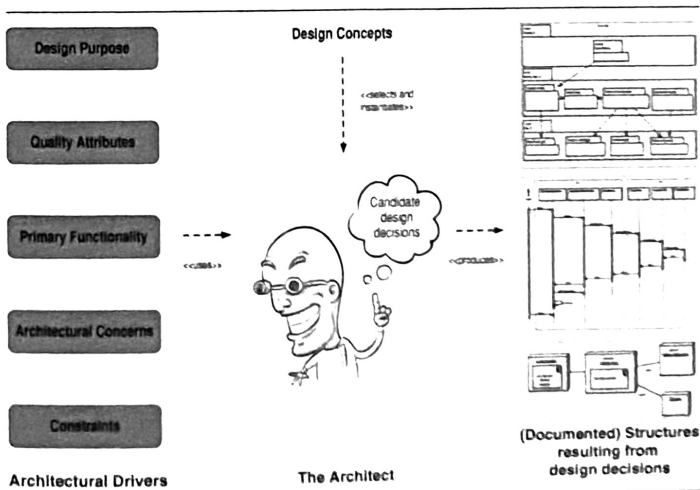


FIGURE 2.1 Overview of the architecture design activity
(Architect image © Brett Lamb | Dreamstime.com)

Bild: Designing Software Architectures

Lecture: <https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=436536>

Herausforderung Struktur: Big Ball of Mud Anti-Pattern

- Big Ball of Mud = System, das keine erkennbare Softwarearchitektur besitzt

- **Gründe:**

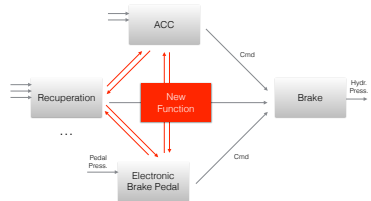
- ungenügende Erfahrung, fehlendes Bewusstsein für Softwarearchitektur
- Erosion

- **Konsequenzen:**

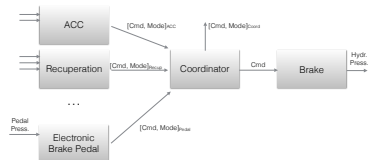
- Mangelnde Wartbarkeit
- Teure Releases
- Minimale Erweiterungen nur mit massivem finanziellen Aufwand

Beispiel:

Koordination verschiedener Assistenzsysteme



VS.



Wie unterstützt man geringe Abhängigkeit, geringe Auswirkung von Änderungen und hohe Wiederverwendbarkeit?

Kopplung (coupling)

Menge von Beziehungen zwischen Subsystemen bzw. Maß dafür, wie stark ein Element verbunden ist, Wissen über andere Elemente besitzt oder von anderen Elementen abhängt.

Typische Formen der Kopplung zwischen A und B:

- A besitzt Attribut mit Verweis auf Instanz von B
- A Objekt ruft Dienst eines B Objekts auf
- A hat Methode welche B Instanz referenziert
 - z.B. Parameter, lokale Variable, Rückgabewert
- A ist direkte oder indirekte Subklasse von B
- B ist Schnittstelle und A implementiert diese Schnittstelle

Starke vs. lose Kopplung

Starke Kopplung (schlecht)

- Lokale Änderungen notwendig bei Änderungen in verbundenen Klassen
- Isoliertes Verstehen der Klassen schwierig (big picture)
- Geringere Wiederverwendbarkeit, da Klassenverwendung Anwesenheit weiterer verbundener Klassen benötigt

Lose Kopplung (gut)

- Lose Kopplung = wenig Beziehungen zwischen Subsystemen
- Wenig Beziehungen $\Rightarrow$ geringe Auswirkung von Änderungen und hohe Wiederverwendbarkeit
- Einfache Wartung / lokale Änderung

Wie bündelt man ein Objekt, macht es verständlich, handhabbar und erreicht – ganz nebenbei – eine lose Kopplung?

Kohäsion (cohesion)

Menge von Beziehungen innerhalb eines Subsystems bzw. Maß dafür, wie stark zusammenhängend und gebündelt die Verantwortlichkeiten im Element sind.

- **Sehr niedrige Kohäsion:** Baustein bietet vollständig unterschiedliche Funktionalität
- **Niedrige Kohäsion:** Klasse hat eine Verantwortlichkeit für eine komplexe Aufgabe in einer komplexen Funktionalität, z.B. eine einzelne Schnittstelle für alle Datenbankzugriffe.
- **Mittlere Kohäsion:** Leichtgewichtige Klasse verantwortlich für einige wenige logische Bereiche, z.B. *Unternehmen* kennt Mitarbeiter und Finanzdetails.
- **Hohe Kohäsion:** Klasse mit moderater Verantwortlichkeit in einem funktionalen Bereich, welche mit anderen Klassen zusammenarbeitet.

Geringe vs. hohe Kohäsion

Geringe Kohäsion

Element mit geringer Kohäsion bietet verschiedene, unzusammenhängende Funktionalitäten. Solche Elemente sind unerwünscht und erzeugen folgende Probleme:

- schwer verständlich
- schwer wiederzuverwenden
- schwer zu warten
- anfällig, andauernd von Änderungen beeinflusst

Hohe Kohäsion

Element mit hohen verbundenen Verantwortlichkeiten. Um hohe Kohäsion aufzuweisen, muss eine Klasse:

- wenige Methoden besitzen,
- wenige Codezeilen groß sein,
- nicht zu viel Funktionalität bieten,
- eine hohe Abhängigkeit des Programmcodes besitzen.

Eigenschaften einer guten Architektur



■ Minimieren der Kopplung zwischen Modulen

- Ziel: Module müssen wenig über Interaktion mit anderen Modulen wissen
- Lose Kopplung erleichtert zukünftige Änderungen

■ Maximieren der Kohäsion in Modulen

- Ziel: Inhalte jeder Komponente sollten stark zusammenhängend sein
- Hohe Kohäsion erleichtert das Verstehen einer Komponente

Oft: Zielkonflikt!

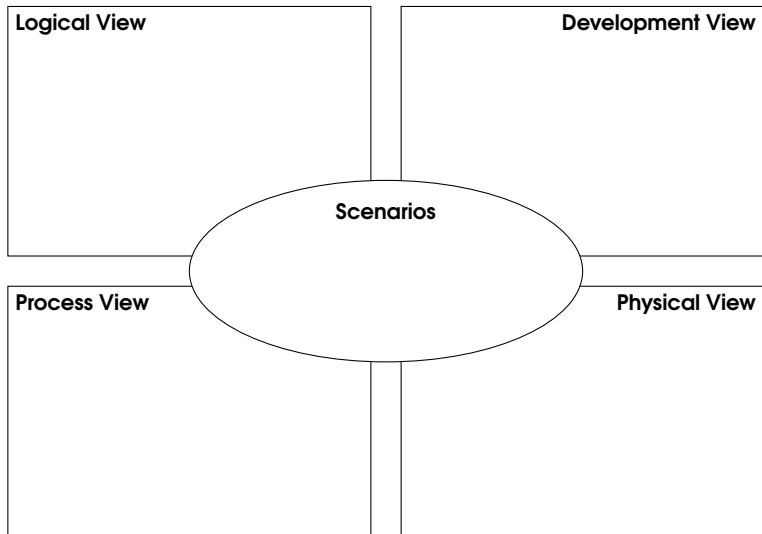
Agenda

1. Motivation
2. Begriffe und Definitionen
- 3. Dokumentation: 4+1 Sichten**
4. Verteilte und Datenintensive Anwendungen
5. Agiles Architektur Design: ADD 3.0
6. Zusammenfassung
7. Literatur und Standards

Dokumentation von Architektur

- Aus den Zielen schrittweise operationelle **Szenarien** abzuleiten und die Architektur gegen diese Szenarien zu prüfen.
- Zur frühzeitigen Identifikation möglicher Risiken oder Probleme sollte die Dokumentation von Software-Architekturen die **Prüfbarkeit** der gewählten Lösung **hinsichtlich der an das System gestellten Anforderungen** unterstützen.
- **Unterschiedliche Architektursichten** ermöglichen durch Walkthroughs die Prüfung funktionaler und nicht-funktionaler Anforderungen anhand von Bewertungs- oder Prüfscenarien.

4+1 Sichten auf eine Architektur



Szenarios

- Anwendungsfälle aus Anforderungsdokumenten
- Grundlage für Prüfung der Architektur:
 - Beschreibung präzise und vollständig?
 - Werden Ziele erreicht?
- **Inhalt:**
 - Abläufe im System für bestimmte Anwendungsfälle
- **Diagramme:**
 - Anwendungsfalldiagramme
 - Textuelle Beschreibung von Anwendungsfällen

Logical View (Funktionale Sicht bzw. Bausteinsicht)

- **Kernstück der Software-Architektur, entspricht dem Grundrissplan eines Hauses**
 - Zeigt statische Struktur des Systems
 - Beschreibt wie das System aus Einzelteilen konstruiert ist
- **Inhalt:**
 - Aufbau aus Softwarebausteinen
 - deren Beziehungen und
 - Schnittstellen
- **Diagramme:**
 - Komponentendiagramm
 - Klassendiagramm (→ SWT)

UML Komponentendiagramm

Beschreibt Komponenten, deren Aufbau, Beziehungen und Schnittstellen

■ Hierarchie:

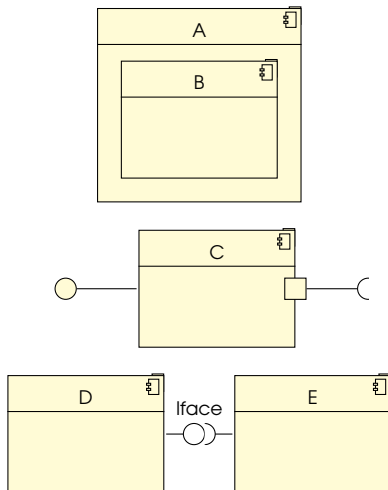
- Komponenten können durch Komponenten detailliert werden

■ Schnittstellen:

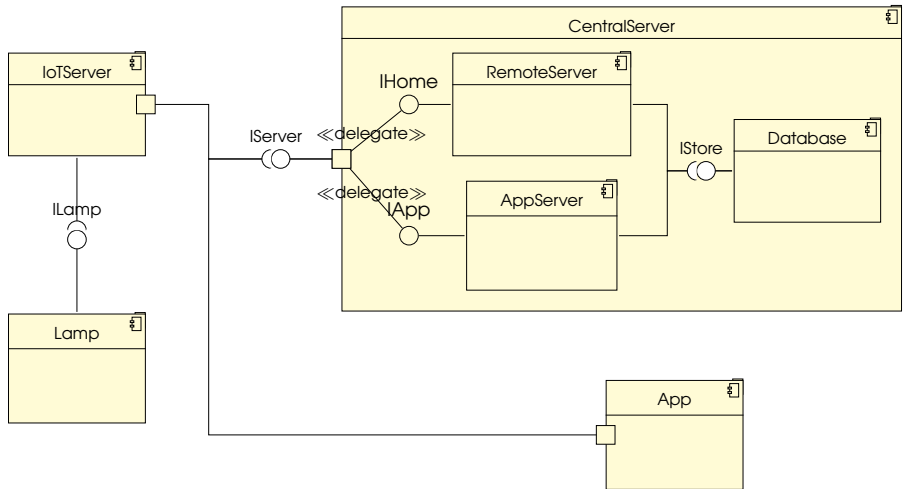
- Es gibt bereitgestellte und benötigte Schnittstellen
- Ports können Schnittstellen zusammenfassen

■ Komposition:

- Komponenten werden über passende Schnittstellen verbunden



Logical View: Beispiel



Development View (Sicht der Entwickler)

■ **Zeigt das System aus Sicht des Entwicklers:**

- Welche Module, IDE-Projekte, Code Repositories gibt es
- Wie wird das System aus diesen Projekten erzeugt

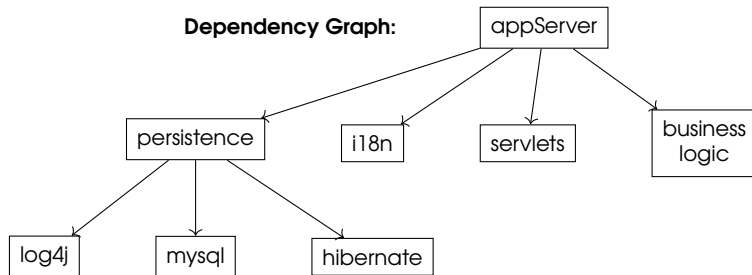
■ **Inhalt:**

- Codegenerierung: Welche Teile des Quellcodes werden generiert und aus welchen Artefakten?
- Build-Management: Wie wird das System aus seinem Quellcode erzeugt?
- Abhängigkeiten

■ **Diagramme:**

- Paketdiagramm
- Abhängigkeitsgraph (nicht UML)
- Build Dateien / Modul Meta-Informationen (nicht UML)

Development View: Beispiel



```
<groupId>gov.nasa</groupId>
<artifactId>jconstraints</artifactId>
<version>0.9.2-SNAPSHOT</version>
<packaging>jar</packaging>
<name>jConstraints</name>
<url>https://github.com/psycopaths/jconstraints</url>
<description>A library for modeling expressions and for interacting with constraint solvers</description>
```

Ausschnitt aus Maven POM

Process View (Dynamische Sicht)

- **Zeigt die Laufzeitsicht das Verhalten des Systems bzw. einzelner Bausteine zur Laufzeit:**
 - wie die Bausteine des Systems zur Laufzeit miteinander bzw. mit den Nachbarsystemen interagieren
 - wie Bausteine des Systems die wesentlichen Funktionen oder Anwendungsfälle erledigen
- **Inhalt:**
 - Wesentliche, beispielhafte Abläufe
- **Diagramme:**
 - Zustandsdiagramm (→ Kap. 3-Modellierung)
 - Aktivitätsdiagramm (→ SWT)
 - Sequenzdiagramm (→ SWT)

Process View: Beispiel

Physical View (Verteilung)

- **Zeigt die Verteilung von Systembestandteilen auf Hard- und Softwareumgebungen:**
 - welche Teile des Systems auf welchen Rechnern, an welchen geographischen Standorten oder in welchen Umgebungen ablaufen
- **Inhalt:**
 - Zuordnung von Artefakten zu Laufzeitumgebungen und Rechnern
 - Ggf. Zuordnung Artefakte / Komponenten
 - Kommunikationsverbindungen
- **Diagramme:**
 - Verteilungsdiagramm

UML Verteilungsdiagramm

Beschreibt Verteilung von Artefakten auf Geräte

■ Orte für Artefakte:

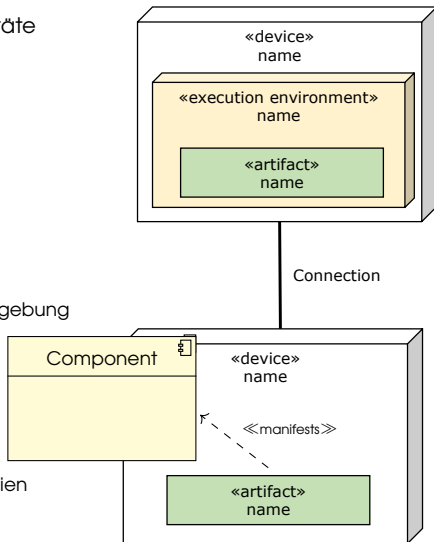
- Device: physikalisches Gerät
- Execution Environment: Logische Ausführungsumgebung

■ Artefakte:

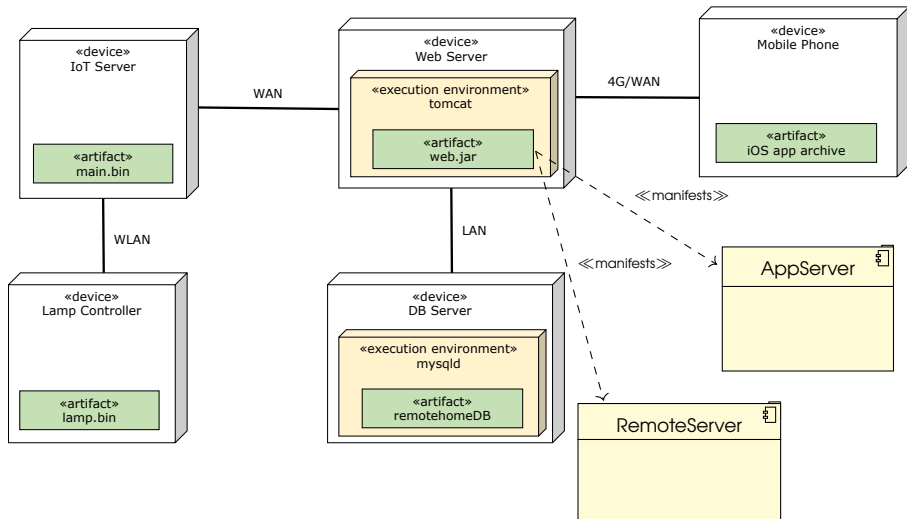
- Auslieferungsgegenstände der Software
- Laufen auf Gerät oder in Ausführungsumgebung
- Manifestieren Komponenten zur Laufzeit

■ Kommunikation:

- Zwischen Geräten
- Angabe konkreter Protokolle / Technologien möglich



Physical View: Beispiel



Agenda

1. Motivation
2. Begriffe und Definitionen
3. Dokumentation: 4+1 Sichten
4. Verteilte und Datenintensive Anwendungen
5. Agiles Architektur Design: ADD 3.0
6. Zusammenfassung
7. Literatur und Standards



Motivation: Twitter

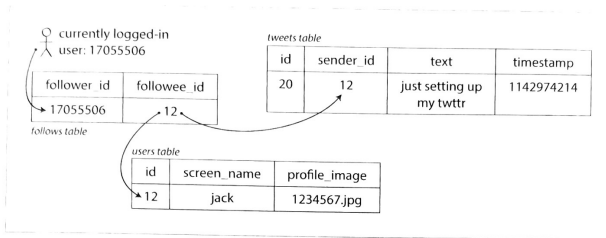


Figure 1-2. Simple relational schema for implementing a Twitter home timeline.

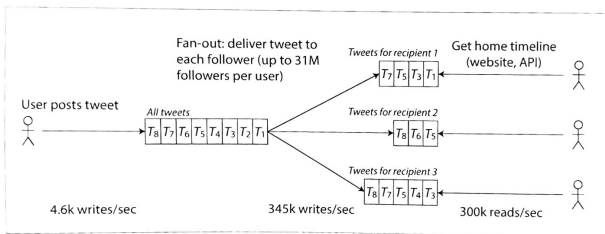
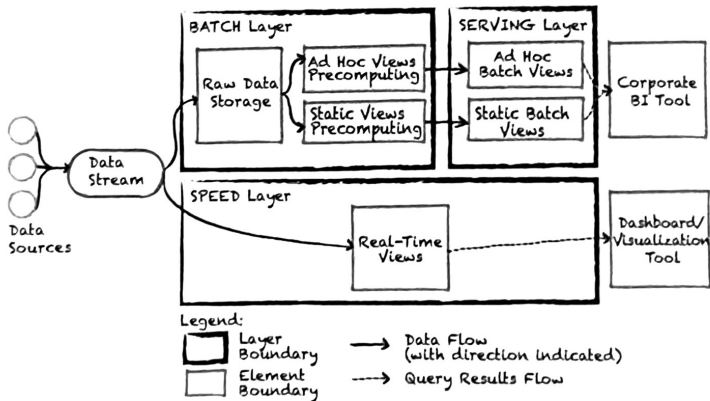


Figure 1-3. Twitter's data pipeline for delivering tweets to followers, with load parameters as of November 2012 [16].

Wir schauen uns nur einen kleinen Teil an!

- Batch-Verarbeitung: Technisches Beispiel für Umgang mit Daten + Praktischer Skill
- Integration Patterns
- Pattern für Skalierbarkeit

Herausforderung: Batch Prozessierung



Arch. Stil: Pipes und Filter



Beispiele:

- Unix Pipes
- Compiler
- Regelungstechnische Systeme

Wichtige Eigenschaften:

- Filter müssen nichts über verbundene Elemente wissen
- Filter können parallel ausgeführt werden
- Jeder Filter arbeitet auf Datenstrom

Implementierung: Unix Batch Processing

Beispiel: Logs analysieren

Analyse:

```
cat server.log |  
  awk '{print $2}' |  
  sort |  
  uniq -c |  
  sort -r -n |  
  head -n 5
```

Log:

```
00:01:09 home.html  
00:03:09 home.html  
...  
00:07:01 conact.html  
...
```

Ausgabe:

```
1231 home.html  
920  contact.html  
...  
1   help.html
```

Pattern / Frameworks: Map Reduce

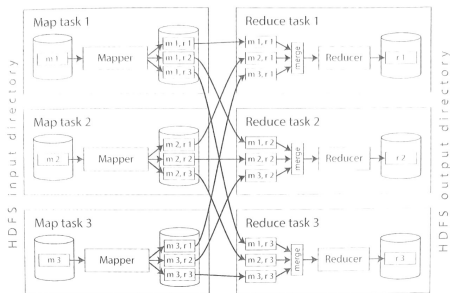


Figure 10-1. A MapReduce job with three mappers and three reducers.

- Frameworks: Hadoop, Spark, ...
- Map (Sortieren + Feature extrahieren)
- Reduce (Aggregation von Daten)
- Jobs verteilt über Knoten

Spark Beispiel (Word count):

```
textFile
  .flatMap(s -> Arrays.asList(s.split(" "))
    .iterator())
  .mapToPair(word -> new Tuple2<>(word, 1))
  .reduceByKey((a, b) -> a + b);
```

Map Reduce Beispiel: Datenanalyse

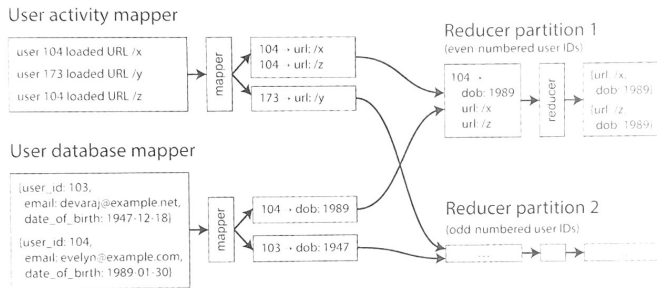
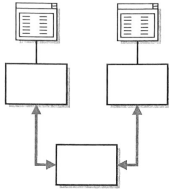


Figure 10-3. A reduce-side sort-merge join on user ID. If the input datasets are partitioned into multiple files, each could be processed with multiple mappers in parallel.

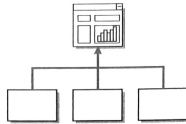
- Integration von URLs und Alter
- Z.B. welche Altersgruppe spielt welche Bretter auf BoardGames?

Integration

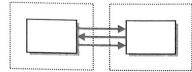
Integration - Szenarios



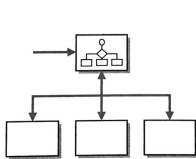
Shared Business Function



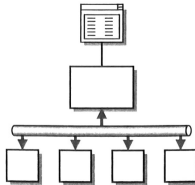
Information Portal



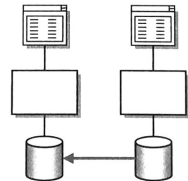
Business-to-Business Integration



Distributed Business Process

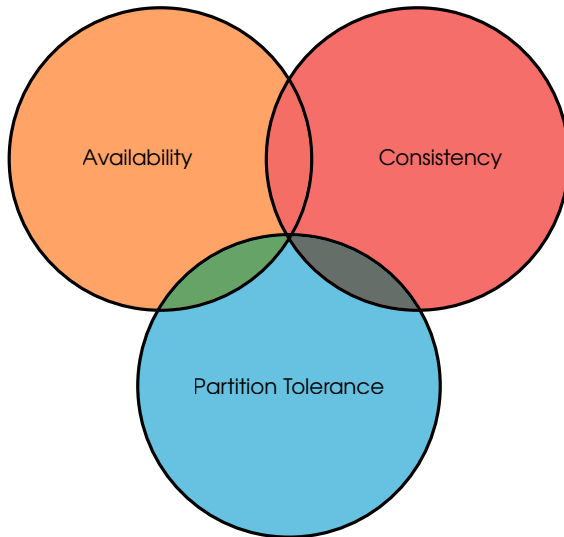


Service-Oriented Architecture



Data Replication

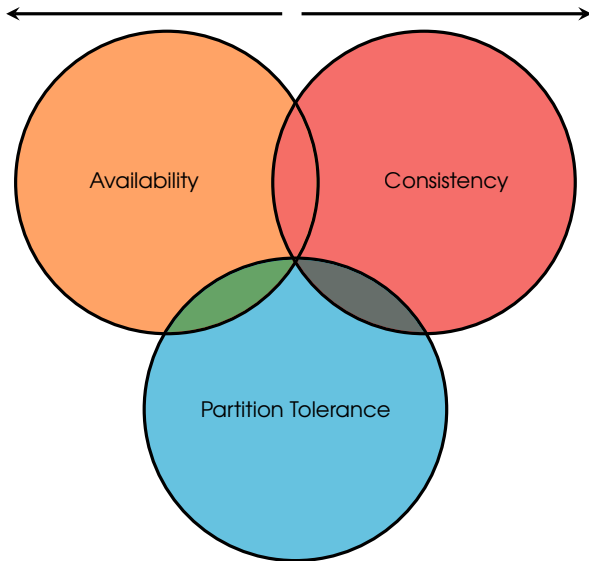
CAP-Theorem



CAP-Theorem (2)

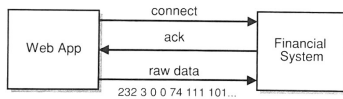
„In larger distributed-scale systems, network partitions are a given; therefore, consistency and availability cannot be achieved at the same time.“

CAP-Theorem (3)



Enge Kopplung vs. lose Kopplung

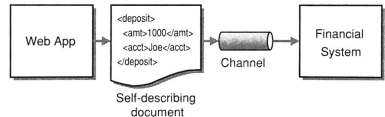
Enge Kopplung



Tightly Coupled Interaction

- Keine Verfügbarkeit bei Partitionierung
- Konsistenz: "immediately"
- Oft: schnellere Antwortzeit
- Skalierung schwieriger
- Verzahnte Entwicklung + Konfiguration

Lose Kopplung



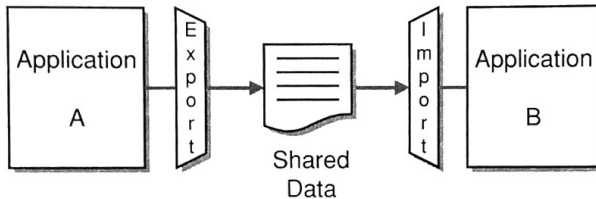
Loosely Coupled Interaction

- Verfügbarkeit bei Partitionierung
- Konsistenz: "eventually"
- Oft: langsamere Antwortzeit bei einzelnen Aufrufen
- Skaliert einfacher
- Unabhängige Entwicklung + Konfiguration

Enterprise Integration Pattern

- Geteilte Datei
- Geteilte Datenbank
- Remote Procedure Call (RPC)
- Nachrichten-basierte Integration

Geteilte Datei



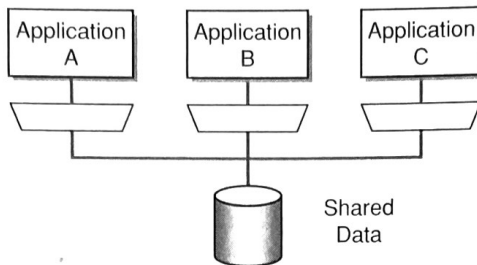
Beschreibung

- Export bei A in Datei
- Import bei B aus Datei
- Beispiel: Upload von Datendatei per FTP

Nutzen/Kosten

- (+) Sehr lose Kopplung
- (+) Stabiles Interface
- (-) Spezifikation Datenformat
- (-) Benötigt Export / Import

Geteilte Datenbank



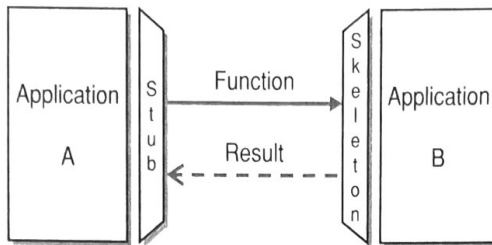
Beschreibung

- Alle Anwendungen teilen Datenbank
- Beispiel: Anbindung von Business Intelligence an bestehende Infrastruktur

Nutzen/Kosten

- (+) Konsistenz
- (+) Einfache Integration
- (-) Skalierung / Partitionierung
- (-) Gemeinsames Schema

Remote Procedure Call - RPC



Beschreibung

- Synchrone Kommunikation
- Enge Kopplung (Kontrollfluss)
- Beispiel: REST-APIs

Nutzen/Kosten

- (+) Konsistenz
- (+) Nah am "normalen" Kontrollfluss
- (-) Partitionierung
- (-) API Entwicklung

REST - Representational State Transfer

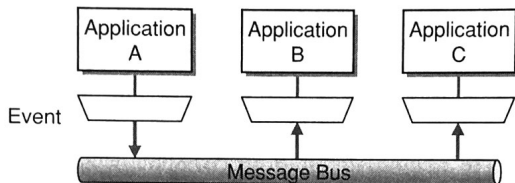
- REST-Paradigma von Roy Fielding 1994 als rationale Rekonstruktion des HTTP Object Models: GET, POST, ...
- Einheitlicher Architekturstil für Web-Services
- **Prinzipien:**
 - Client-Server
 - Zustandslosigkeit
 - Caching
 - Einheitliche Schnittstelle
 - Mehrschichtige Systeme
 - Code on Demand (optional)



Beispiel: Google Drive

get	GET /files/ fileId	Gets a file's metadata by ID.
insert	POST https://www.googleapis.com/upload/drive/v2/files and POST /files	Insert a new file.
patch	PATCH /files/ fileId	Updates file metadata. This method supports patch semantics .
update	PUT https://www.googleapis.com/upload/drive/v2/files/ fileId and PUT /files/ fileId	Updates file metadata and/or content.
copy	POST /files/ fileId /copy	Creates a copy of the specified file.
delete	DELETE /files/ fileId	Permanently deletes a file by ID. Skips the trash. The currently authenticated user must own the file or be an organizer on the parent for Team Drive files.
list	GET /files	Lists the user's files.
touch	POST /files/ fileId /touch	Set the file's updated time to the current server time.
trash	POST /files/ fileId /trash	Moves a file to the trash. The currently authenticated user must own the file or be an organizer on the parent for Team Drive files.
untrash	POST /files/ fileId /untrash	Restores a file from the trash.
watch	POST /files/ fileId /watch	Start watching for changes to a file.
emptyTrash	DELETE /files/trash	Permanently deletes all of the user's trashed files.
generateIds	GET /files/generateIds	Generates a set of file IDs which can be provided in insert requests.
export	GET /files/ fileId /export	Exports a Google Doc to the requested MIME type and returns the exported content. Please note that the exported content is limited to 10MB.

Nachrichten-basierte Integration



Beschreibung

- Asynchrone Kommunikation
- Lose Kopplung (Kanäle / Broker)
- Beispiel: MQTT

Nutzen/Kosten

- (+) Partitionierung
- (+) Skalierung / Konfiguration
- (-) Datenfluss vs. Kontrollfluss
- (-) Konsistenz

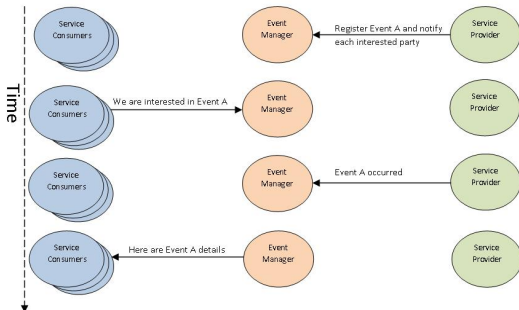
Beispiel: Publish/Subscribe Message Broker

Wichtige Eigenschaften:

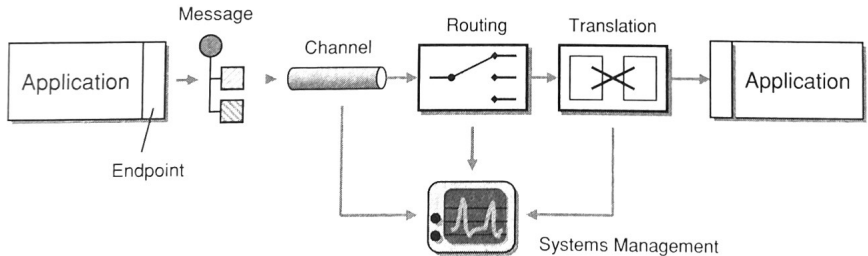
- Ereignisquelle muss Ereignissenken nicht kennen (impliziter Aufruf)
- Unterstützt Wiederverwendung und Evolution

Herausforderungen:

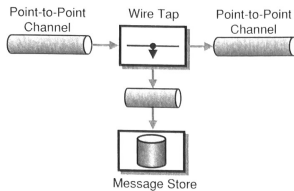
- Komponenten haben keine Kontrolle von Berechnungsreihenfolge
- Garantie für 1x Auslieferung schwierig



Etwas Detaillierter: Bausteine



Basic Elements of Message-Based Integration

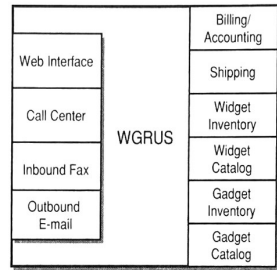
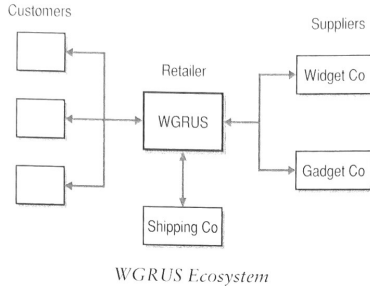


Tracking Messages with a Wire Tap

(Basis-)Bestandteile der Architektur

- **Endpunkt/Adapter:** Komponente, die Anwendungen mit Nachrichten-System verbindet
- **Nachricht:** Strukturierte, selbst-beschreibende Informationseinheit, die gesendet / empfangen / verarbeitet wird
- **Punkt-2-Punkt Kanal:** Verbindung zwischen zwei Komponenten, die Nachrichten bearbeiten
- **Publish/Subscribe Kanal:** Themen-basierter Kanal
- **Router:** Komponente, die Nachrichten aus einem Kanal empfängt und regel-basiert in Kanäle weitersendet
- **Kanal-Abhörer:** Spezieller Router, der Nachrichten auf einem Kanal
- **Übersetzer:** Transformiert Nachrichten von einem Format in ein anderes Format
- **Anreicherer:** Fügt Nachrichten weitere Informationen hinzu
- **Aggregator:** Integriert Nachrichten aus verschiedenen Kanälen
- **Nachrichten Speicher:** Speichert Nachrichten permanent
- **Toter Kanal:** Zerstört Nachrichten

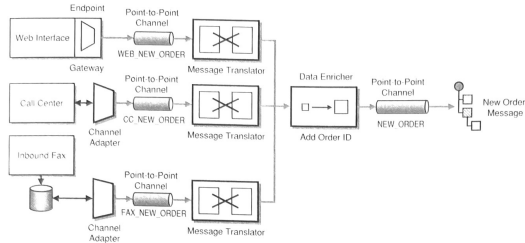
Beispiel: Widgets'R'Us



WGRUS IT Infrastructure

- Verschiedene IT Systeme
- Netz von Zulieferern und Logistik Partnern
- Kunden nutzen verschiedene Kommunikationskanäle

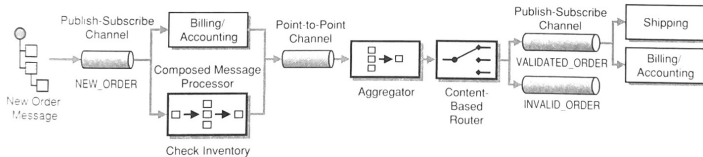
Widgets'R'Us: Bestellungseingang



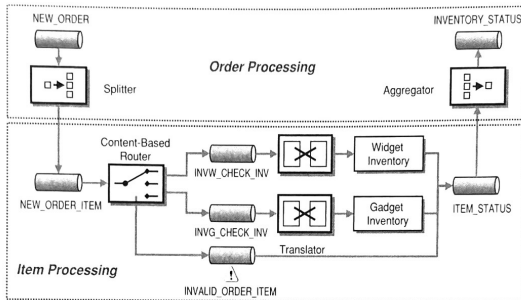
Taking Orders with Enricher

- Verschiedene Systeme senden Daten an ein System
- Weitere Front-ends: Mehr Sender

Widgets'R'Us: Bestellung zusammenstellen

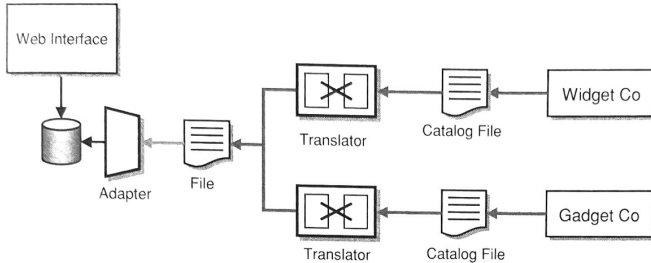


Revised Order Process Implementation



Processing Order Items Individually

Widgets'R'Us: Update Warenkatalog



Updating Catalog Data via File Transfer

- Zwischen verschiedenen Unternehmen → Wenig Zugriff ermöglichen
- Einmal am Tag / in der Nacht ausreichend
- Pattern: Geteilte Datei

Herausforderungen

- **Globaler Zustand:** In welchem Zustand ist die Anwendung bzw. eine Bestellung? Schwierig feststellen. Einzelne Anwendungen möglichst unabhängig, Nachrichten nicht fein-granular.
- **Blockaden:** Falsch konfiguriert liegen Nachrichten zu lange vor einem Aggregator bzw. woher weiß ein Aggregator, wie viele Teile zusammengehören?
- **Garantie der Zustellung:** Es ist schwer **genau einmal** zu garantieren (z.B. Systemabstürze, Partitionierung, Publish/Subscribe) oft stattdessen: **mindestens einmal**

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Dokumentation: 4+1 Sichten
4. Verteilte und Datenintensive Anwendungen
5. Agiles Architektur Design: ADD 3.0
6. Zusammenfassung
7. Literatur und Standards

Architektur Lebenszyklus

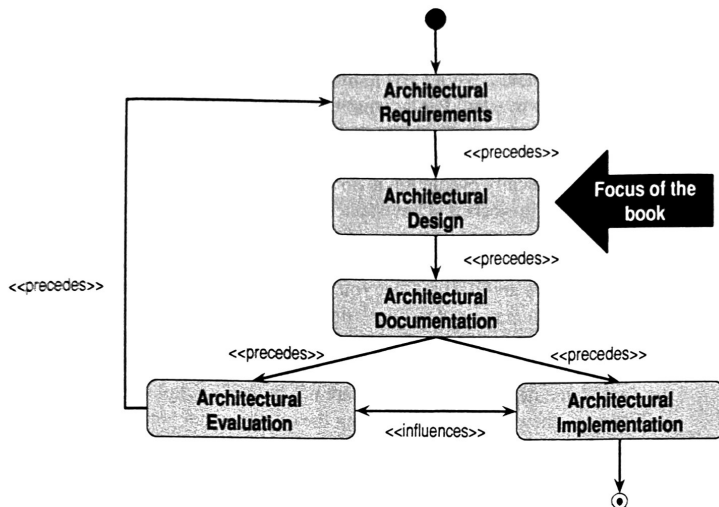


FIGURE 1.1 Software architecture life-cycle activities

Attribute-Driven Design 3.0 (ADD)

- Iteratives Vorgehensmodell zum Architektur Design
- Entwickelt am Software Engineering Institute (SEI) von CMU
- Fokus auf einzelne Design Entscheidungen

Andere Vorgehensmodelle:

- RUP
- Architecture-Centric Design Method
- The Process of Software Architecting
- Application Architecture Guide
- Viewpoints and Perspectives

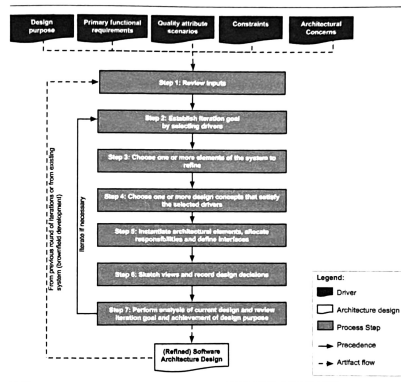


FIGURE 3.1 Steps and artifacts of ADD version 3.0

Architektur Treiber

- Design Ziel
- Qualitätsanforderungen
- Funktionalität
- Anforderungen an die Architektur
- Randbedingungen

Design Ziel: Why?

Was ist das Ziel des Design Prozesses?

- Design Architektur neues System für X
- Integration von Anwendung Y in Unternehmenskontext
- Prototyp für Anwendung in neuem Paradigma
- Re-Strukturierung bestehendes System

Qualitätsanforderungen

Definition (Qualitätsanforderung)

Eine Qualitätsanforderung ist eine Anforderung, die sich auf ein Qualitätsmerkmal bezieht, das nicht durch funktionale Anforderungen abgedeckt wird.

Beispiele:

■ Performance:

"Das System muss jede Anfrage des Benutzers innerhalb von 30 Sekunden ausführen (auf System XY)."

"Der Speicherbedarf darf 512MB nicht übersteigen."

■ Zuverlässigkeit:

"Die Verfügbarkeit des Systems muss bei 99.999% liegen."

■ Benutzbarkeit:

„Das System muss die gespeicherten Objekte formatiert ausgeben können nach Schema XXXX.“

Definition (Funktionale Anforderung)

Eine funktionale Anforderung ist eine Anforderung an das Verhalten, das von einer Funktion des Systems bereitgestellt werden soll.

Beispiele:

- *"Der Benutzer kann sich mit seinem Nutzernamen oder seiner E-Mail Adresse anmelden."*
- *"Das System zeigt in der Übersicht zu jedem Gerät im Haus den Status und den Zeitstempel der letzten Interaktion an."*

Architekturelle Rahmenbedingungen

Definition (Architekturelle Rahmenbedingungen)

Architekturelle Rahmenbedingungen schränken den Lösungsraum für eine Architektur jenseits dessen ein, was notwendig ist, um die funktionalen Anforderungen und die Qualitätsanforderungen zu erfüllen.

Beispiele:

- *„Zur Authentifizierung wird OAuth verwendet.“*
- *„Die App wird auf einer Single-Core CPU deployed.“*
- *„Integration von System A muss über MQTT erfolgen.“*
- *„Das System muss mit Java entwickelt werden und muss in der Sun Java VM Version XY laufen.“*

Randbedingungen

Definition (Randbedingungen)

Eine Randbedingung ist eine Anforderung, die den Lösungsraum jenseits dessen einschränkt, was notwendig ist, um die funktionalen Anforderungen und die Qualitätsanforderungen zu erfüllen.

Beispiele:

- *„Es darf nur Open-Source Technologie genutzt werden.“*
- *„Es muss ein Cloud Anbieter in Europa gewählt werden.“*

Dokumentation: Szenarios (Perspektive +1)

- Anwendungsfälle aus Anforderungsdokumenten
- Grundlage für Prüfung der Architektur:
 - Beschreibung präzise und vollständig?
 - Werden Ziele erreicht?
- Abläufe im System für bestimmte Anwendungsfälle

Abschnitt	Inhalt
Bezeichner	UC-1
Name	Gerät schalten
Autoren	F. Howar
Priorität	...
Quelle	Nutzerbefragung
Verantwortlich	...
Kurz-Beschreibung	Der Nutzer schaltet ein Gerät im Haus von unterwegs ein.
Auslöser	Der Wunsch, ein Gerät zu schalten.
Vorbedingung	App installiert und Gerät registriert.
Nachbedingung	
Ergebnis	
Hauptszenario	App öffnen, Gerät aus Übersicht wählen, Gerät schalten
Alternativ-Szenarien	...
Ausnahme-Szenarien	...
Einzelne Anforderungen	REQ-F-10, ...
Qualitäten	REQ-Q-1, REQ-Q-2, ...

Design Konzepte

- Referenzarchitekturen
- Pattern
- Taktiken
- Extern entwickelte Komponenten

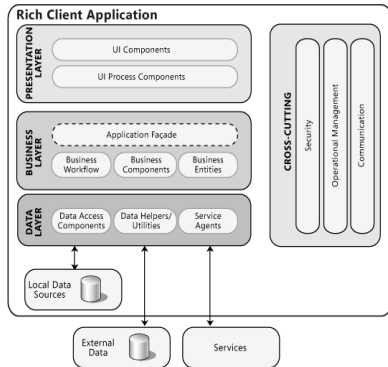
Referenzarchitekturen

Baupläne/Vorlagen für eine Software Architektur in einem speziellen Kontext (kann alle Perspektiven beinhalten).

Beinhaltet typischerweise: Konkrete Schichten, Kommunikation, konkrete Komponenten und Kommunikationsbeziehungen, Deployment

Beispiele:

- Schichten-Architektur u. LAMP von Web-Anwendungen
- Load-Balancing Architekturen (vgl. SWK)
- Sense-Plan-Act im Bereich autonomes Fahren



Quelle: <https://msdn.microsoft.com/en-us/library/ee658107.aspx>

siehe Folien zu diesem Thema!

Taktiken

Taktiken sind Designs und Techniken, die auf die Erfüllung oder Verbesserung **eines** Qualitätsattributes abzielen.

Im Vergleich zu Pattern: eingeschränkter Fokus auf eine Technik die ein Attribut (positiv) beeinflusst

Attribute:

- Availability
- Interoperability
- Modifiability
- Performance
- Security
- Testability
- Usability

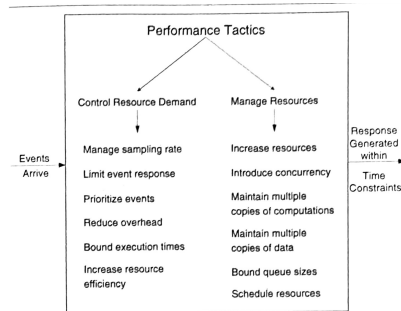


FIGURE 2.9 Performance tactics from *Software Architecture in Practice*

Extern entwickelte Komponenten

- **Technologien:** *Familien von Komponenten:* ORM, Generative Programmierung, CI, Datenbank, ...
- **Produkte:** *Konkrete Software Produkte:* SQL Server, Ubuntu, ...
- **Frameworks:** *Wiederverwendbare Elemente (basierend auf Pattern und Taktiken):* Tomcat, Hibernate, ...
- **Platforms:** *Infrastruktur:* Java, .Net, AWS, Azure, ...

Schritt 1: Review Inputs

Ziel:

In diesem Schritt ist zu prüfen, dass alle benötigten Artefakte vorhanden sind und ist die Qualität / Klarheit ausreichend ist

Fragen:

- Ist das **Problem** verstanden?
- Sind Anforderungen verständlich?

Risiko:

“Garbage In - Garbage Out”

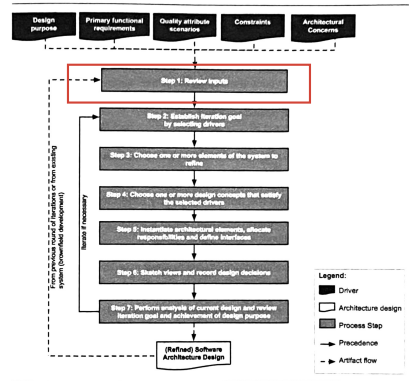


FIGURE 3.1 Steps and artifacts of ADD version 3.0

Schritt 2: Ziel für Iteration durch Wahl von Treibern

Ziel:

In diesem Schritt wird ein Teil des Design-Problems ausgewählt der in der aktuellen Iteration gelöst oder verbessert werden soll.

Voraussetzung:

Das Design-Probleme ist in Teile / Aspekte zerlegt (z.B., einzelne Szenarien / Anforderungen)

Vorgehen:

- 1 Allgemeine Struktur
- 2 Primäre Funktionen
- 3 Qualität / Weitere

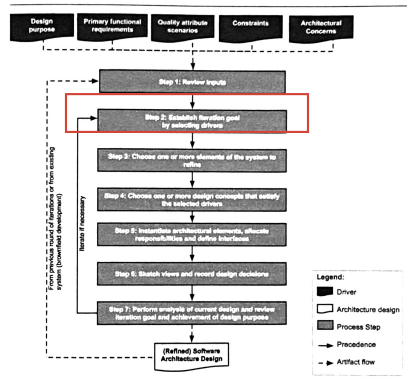


FIGURE 3.1 Steps and artifacts of ADD version 3.0

Schritt 3: Auswahl zu verbessernde Elemente

Ziel:

Auswahl, welche Teile des bereits existierenden Architektur Entwurfs bearbeitet bzw. zerlegt werden in dieser Iteration

Vorgehen:

- Z.B. Szenarios
- Siehe nächste Folie

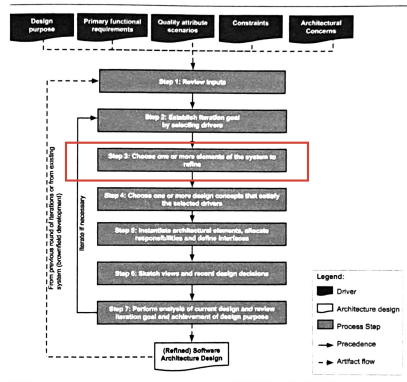


FIGURE 3.1 Steps and artifacts of ADD version 3.0

Beispiel: Vorgehen für Green Field Entwicklungen

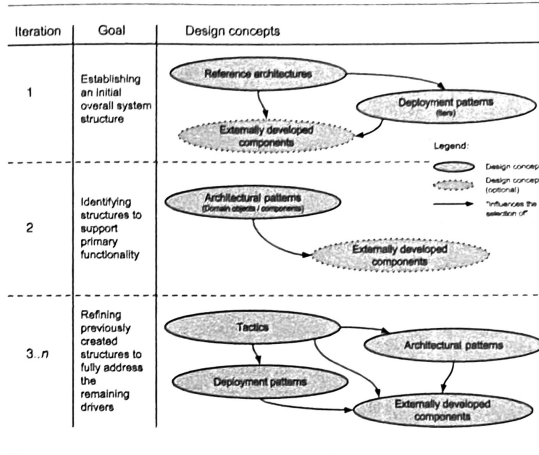


FIGURE 3.2 Design concept selection roadmap for greenfield systems

→ Einfluss

Schritt 4: Auswahl Design Konzepte für Treiber

- Kandidaten für verschiedene Konzepte
- Auswahl eines Konzepts

(siehe Design Entscheidungen)

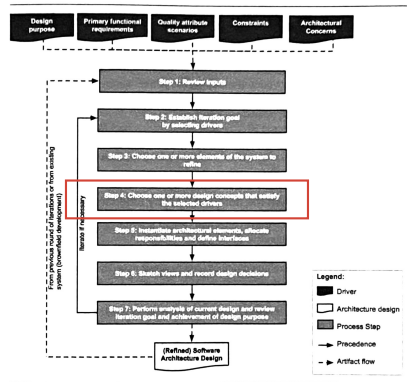
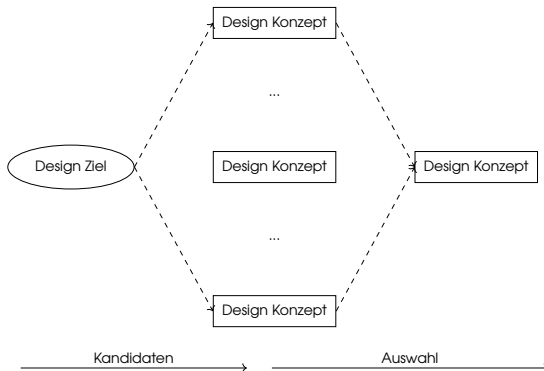


FIGURE 3.1 Steps and artifacts of ADD version 3.0

Architektur Design Entscheidungen



- Verschiedene Konzepte führen zu verschiedenen Lösungen
- Entscheidungen **quantitativ** wenn möglich
- Prototypen können helfen
- Größere Risiken früh entscheiden

Schritt 5: Konzepte anwenden

- Konzepte konkret instanzieren
- ggf. Aufgaben/Funktion zuordnen
- ggf. Schnittstellen / Kommunikation planen

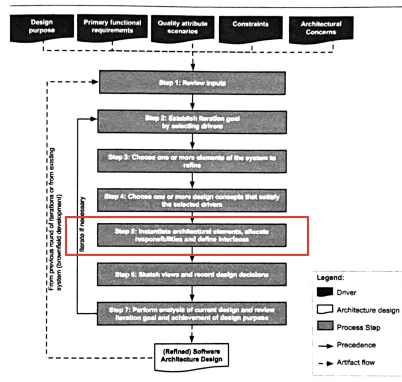
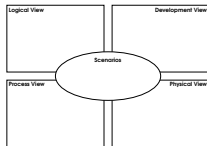


FIGURE 3.1 Steps and artifacts of ADD version 3.0

Schritt 6: Views modellieren + Entscheidungen dokumentieren



- Dokumentation wird inkrementell formaler und vollständiger
- Gründe für Entscheidungen (bzw. für / gegen Konzepte) dokumentieren!
- Daten / Prototypen ablegen

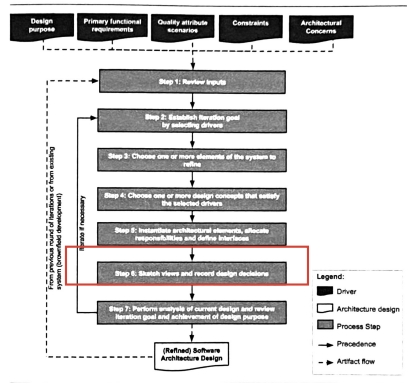


FIGURE 3.1 Steps and artifacts of ADD version 3.0

Schritt 7: Analyse der Lösung

Ziel:

Validieren gefundener Lösung

Vorgehen:

- **Szenarios:** Manuelles durchspielen von Szenarios
- **Reviews:** Überprüfung der gefundenen Lösung durch Stakeholder oder Dritte
- **Tests:** Evtl. gibt es Performanz Tests, Qualitäts Test, ...

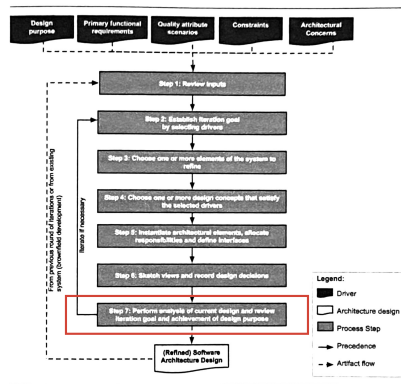


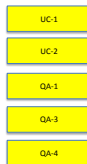
FIGURE 3.1 Steps and artifacts of ADD version 3.0

Iteration

Frage: Wie viel Design ist genug Design?

- Priorisierung von Treibern vor Design
- Sind alle Treiber adressiert?
- Werkzeug z.B. Design Kanban Board

Not addressed



Partially addressed

Addressed



Beispiel: Remote Home Control

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Dokumentation: 4+1 Sichten
4. Verteilte und Datenintensive Anwendungen
5. Agiles Architektur Design: ADD 3.0
- 6. Zusammenfassung**
7. Literatur und Standards

Zusammenfassung

Architektur

- Umsetzbarer Entwurf der eines Systems
- Bauplan

Dokumentation

- 4+1 Sichten

Qualität

- Kopplung
- Kohäsion

Vorgehen

- Treiber, Ziele, Prinzipien
- Herausforderungen
- Iteratives Vorgehen: ADD 3.0

Muster

- Etablierte Lösungsmuster
- Adressieren Herausforderungen

Beispiele

- Datenverarbeitung
- Verteilung
- Integration

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Dokumentation: 4+1 Sichten
4. Verteilte und Datenintensive Anwendungen
5. Agiles Architektur Design: ADD 3.0
6. Zusammenfassung
7. Literatur und Standards

- Hruschka, Peter und Starke, Gernot. Software-Architektur kompakt: angemessen und zielorientiert, Spektrum, 2011.
<https://dx.doi.org/10.1007/978-3-8274-2835-6>
- Duggan, Dominic. Enterprise software architecture and design : entities, services and resources, Wiley, 2012.
<https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781118180518>
- Gregor Hohpe and Bobby Wolf: **Enterprise Integration Patterns**. Addison-Wesley, 2004.
- Martin Kleppmann: **Designing Data-Intensive Applications**. O'Reilly, 2017.
- Humberto Cervantes and Rick Kazman: **Designing Software Architectures**. Addison-Wesley, 2016.
- Philippe Kruchten: **Architectural Blueprints - The 4+1 View Model of Software Architecture**. IEEE Software 12 (6), pp. 42-50, 1995. (UB/Google)

Standards

- **ISO/IEC/IEEE 42010** Systems and software engineering — Architecture description
- **OMG 2007** OMG: Unified Modeling Language: Superstructure, Version 2.1.1. OMG document formal/2007-02-05.